EGE UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

INTRODUCTION TO OBJECT ORIENTED PROGRAMMING

FOOTBALL PLAYER INFORMATION SYSTEM

2024–2025 SPRING SEMESTER

PROJECT REPORT

DELIVERY DATE : 14.06.2025

PREPARED BY

GÖRKEM EGE AKINCI 05220000315

GÖRKEM EGE AKINCI 05220000315

Contents

Project Overview / Abstract 3
Requirements - Implementation Mapping 3
Key Algorithms & Data Structures 5
Implementation Details 7
Testing & Results 9
Time Log 11
Class Explanations 12

GÖRKEM EGE AKINCI 05220000315

Project Overview / Abstract

This project presents a console-based “Football Player Information System” developed for the course *Introduction to Object-Oriented Programming*. The goal is to record and manage football players’ personal data—first name, last name, birth date, and a history of clubs—while maintaining the collection in strict alphabetical order. To satisfy this requirement, the program implements its own **doubly linked list**, providing efficient bidirectional traversal, sorted insertion, search, and deletion operations. Each node stores a Player object plus forward and backward links, enabling fast printing from A→Z or Z→A without additional memory. For “last-in, first-out” functionality the system also introduces an **array-based stack** (MyStack) that holds generic Object items; it supports dynamic resizing, push, pop, and emptiness checks, and is used to demonstrate bulk copy of the list’s contents. All file I/O, date validation (including leap-year checks), and menu-driven user interaction are handled through modular helper classes, ensuring clean code structure and ease of testing.

Requirements - Implementation Mapping

Requirement (from project sheet)	How it is fulfilled in our code
Build doubly linked list from *players.txt* – keep alphabetical order by name → surname.	PlayerFileLoader.buildListFromFile(String) reads the CSV, creates Player objects, and inserts each one with DoublyLinkedList.insertSorted(Player), which maintains alphabetical order automatically.
Add a new player from keyboard in the correct position.	PlayerInputUtil.addPlayerFromKeyboard(Scanner, DoublyLinkedList) collects user data, constructs a Player, and calls insertSorted to place it in order.
Print all information of a	PlayerQueryUtil.printPlayerInfo(Scanner, DoublyLinkedList) prompts for the keys, uses

given player (name + surname).	DoublyLinkedList.search(String,String), and prints the returned Player.toString().
Delete a player by name + surname.	PlayerDeleteUtil.deletePlayer(Scanner, DoublyLinkedList) calls DoublyLinkedList.delete(String,String) and reports success/fail to the console.
Print the list in ascending alphabetical order (A→Z).	PlayerPrintUtil.printAscending(DoublyLinkedList) → DoublyLinkedList.printAscending() traverses from head to tail.
Print the list in descending order (Z→A).	PlayerPrintUtil.printDescending(DoublyLinkedList) → DoublyLinkedList.printDescending() traverses from tail to head.
Copy the list to a stack and print the stack.	PlayerStackUtil.copyListToStackAndPrint(DoublyLinkedList) traverses the list with DoublyLinkedList.Iterator, pushes each Player onto MyStack, then pops and prints in LIFO order.
EXIT: before quitting, write the final list to a temp file.	In Main, option 8 triggers PlayerFileSaver.saveListToFile(DoublyLinkedList,String), which iterates (A→Z) with DoublyLinkedList.Iterator and saves to *players_updated.txt*.

Core data structures used:

- **DoublyLinkedList** (head & tail pointers) for ordered storage and bidirectional traversal.
- **MyStack** (dynamic Object[] array) for requirement 7's LIFO copy–print operation.

Key Algorithms & Data Structures

1 Alphabetical Insertion into the Doubly Linked List

Logic: every time we add a new Player, we compare it to the existing nodes with Player.compareTo():

```
@Override
public int compareTo(Player other) {
    int cmp = this.name.compareToIgnoreCase(other.name);
    return (cmp != 0) ? cmp
                      : this.surname.compareToIgnoreCase(other.surname);
}
```

This guarantees O(n) worst-case time and always preserves perfect A→Z order.

2 Bidirectional Traversal

Because each Node keeps **two links** (prev, next), we can traverse in **either** direction without extra memory:

```
/* ---------- Printing ---------- */
public void printAscending() {            // A → Z
    Node cur = head;
    while (cur != null) {
        System.out.println(cur.data);
        cur = cur.next;
    }
}

public void printDescending() {           // Z → A
    Node cur = tail;
    while (cur != null) {
        System.out.println(cur.data);
        cur = cur.prev;
    }
}
```

This makes menu options 5 and 6 constant-time per element and avoids reversing or copying the list.

3 Dynamic Array Stack (MyStack)

MyStack stores Object[] data, an int top, and an int capacity.
When a **push** would overflow the array, we *double* its size—amortized O(1):

```
private void expandCapacity() {
    capacity *= 2;
    Object[] newArr = new Object[capacity];
    System.arraycopy(data, 0, newArr, 0, top + 1);
    data = newArr;
}
```

Push & Pop core:

```
public void push(Object item) {
    if (item == null)
        throw new IllegalArgumentException("Null item cannot be pushed.");
    if (top == capacity - 1)            // array full -> expand
        expandCapacity();
    data[++top] = item;                 // increment index, then store
}

/**
 * Removes and returns the last added element (pop).
 * Throws IllegalStateException if the stack is empty.
 */
public Object pop() {
    if (isEmpty())
        throw new IllegalStateException("Stack is empty.");
    Object item = data[top];
    data[top] = null;                   // avoid memory leak
    top--;
    return item;
}
```

This simple strategy lets requirement 7 copy any-size list into the stack without pre-calculating capacity, while keeping code easy to understand for beginners.

Implementation Details

---

1) File I/O Format (players.txt)

CSV Layout:

firstName,surname,dd/MM/yyyy,club1,club2,club3,...

Example line:

Fernando,Muslera,16/06/1986,Wanderers,Lazio,Galatasaray

Reading – PlayerFileLoader.buildListFromFile(path)

1. Opens the file with BufferedReader.
2. Splits each non-blank line by commas.
3. Parses dd/MM/yyyy into integers *(day, month, year)* and constructs a validated Date.
4. Collects the remaining tokens into a List<String> of clubs.
5. Creates a Player object and inserts it via DoublyLinkedList.insertSorted().

This guarantees the in-memory list is always alphabetical the moment loading completes.

Writing – PlayerFileSaver.saveListToFile(list, "players_updated.txt")

Iterates A→Z with DoublyLinkedList.Iterator, rebuilds each line in the same CSV format, and flushes to disk with BufferedWriter. Thus the saved file can be re-loaded later without modification.

---

2) Menu Loop & User Interaction (Main)

```java
Scanner kb = new Scanner(System.in);
boolean exit = false;
while (!exit) {
    printMenu();                        // shows the 8 options
    String choice = kb.nextLine().trim();
    switch (choice) {
        case "1": /* load file */           break;
        case "2": /* add player */          break;
        ...
        case "8": /* save & quit */ exit = true; break;
        default : System.out.println("Invalid choice!");
    }
    System.out.println();               // blank line for readability
}
kb.close();
```

Key points

- **Single Scanner** – reused across the whole program to avoid stream clashes.
- **Input validation** – after options 3 & 4 we call kb.nextLine() once to consume the leftover newline, preventing accidental empty reads.
- **Unicode icons** (✅, ⚠, ✕) provide quick visual feedback.
- **Graceful exit** – option 8 triggers PlayerFileSaver, prints a goodbye message, and then breaks out of the loop.

Testing & Results

You can decide which operation will you perform by writing numbers in my code. Each operation has its own number (1,2,3…8). After a successful operation code will ask you again.

```
======= Player Information System =======
1) Create list from file
2) Add player from keyboard
3) Show player information
4) Delete player
5) Print all records A → Z
6) Print all records Z → A
7) Copy list to stack and print
8) Exit (save list)
```

Option 1 - Create list from file

```
Your choice (1-8): 1
File path (leave empty for default: players.txt):
✅ List loaded from file. Total records: 7
```

This means that file was read correctly and **7 player records** were inserted into the doubly linked list.

Option 2 - Add player from keyboard

(You should press enter once before writing the name!)

```
Your choice (1-8): 2

First Name: Görkem
Last Name: Akıncı
Birth date (dd/mm/yyyy): 29/07/2003
Enter clubs separated by commas: Galatasaray,PSG
✅ Player added to the list: Görkem AKINCI (29/07/2003) - [Galatasaray, PSG]
```

You should write the needed information about player and then it adds that player to the list.

Option 3 - Show player information

```
Your choice (1-8): 3
Enter first name to search: Görkem
Surname: Akıncı

🔎 Player found:
Görkem AKINCI (29/07/2003) - [Galatasaray, PSG]
```

This option shows the information about that specific player.

Option 4 – Delete player

```
Your choice (1-8): 4
Name of player to delete: Görkem
Surname: Akıncı
'Görkem Akıncı' successfully deleted from the list.
```

This option deletes the player from the list.

Options 5,6 – Print all records

```
Your choice (1-8): 5

--- Player List (A → Z) ---
Edin DZEKO (17/03/1986) - [Manchester City, Roma, Inter, Fenerbahce]
Fernando MUSLERA (16/06/1986) - [Wanderers, Lazio, Galatasaray]
Rafa SİLVA (17/05/1993) - [Braga, Benfica, Besiktas]
Ricardo QUARESMA (26/09/1983) - [Sporting Lizbon, Inter, FC Porto, Besiktas]
Serdar AZİZ (23/10/1990) - [Bursaspor, Galatasaray, Fenerbahce]
Serdar GURLER (14/09/1991) - [Trabzonspor, Genclerbirligi, Goztepe, Antalyaspor, Basaksehir FK]
Victor OSİMHEN (29/12/1998) - [Wolfsburg, Lille, Napoli, Galatasaray]

======= Player Information System =======
1) Create list from file
2) Add player from keyboard
3) Show player information
4) Delete player
5) Print all records A → Z
6) Print all records Z → A
7) Copy list to stack and print
8) Exit (save list)
==================================
Your choice (1-8): 6

--- Player List (Z → A) ---
Victor OSİMHEN (29/12/1998) - [Wolfsburg, Lille, Napoli, Galatasaray]
Serdar GURLER (14/09/1991) - [Trabzonspor, Genclerbirligi, Goztepe, Antalyaspor, Basaksehir FK]
Serdar AZİZ (23/10/1990) - [Bursaspor, Galatasaray, Fenerbahce]
Ricardo QUARESMA (26/09/1983) - [Sporting Lizbon, Inter, FC Porto, Besiktas]
Rafa SİLVA (17/05/1993) - [Braga, Benfica, Besiktas]
Fernando MUSLERA (16/06/1986) - [Wanderers, Lazio, Galatasaray]
Edin DZEKO (17/03/1986) - [Manchester City, Roma, Inter, Fenerbahce]
```

This option prints all records A-Z or Z-A.

Option 7 – Copy list to stack and print

```
Your choice (1-8): 7

--- Stack (LIFO) Output ---
Victor OSİMHEN (29/12/1998) - [Wolfsburg, Lille, Napoli, Galatasaray]
Serdar GURLER (14/09/1991) - [Trabzonspor, Genclerbirligi, Goztepe, Antalyaspor, Basaksehir FK]
Serdar AZİZ (23/10/1990) - [Bursaspor, Galatasaray, Fenerbahce]
Ricardo QUARESMA (26/09/1983) - [Sporting Lizbon, Inter, FC Porto, Besiktas]
Rafa SİLVA (17/05/1993) - [Braga, Benfica, Besiktas]
Fernando MUSLERA (16/06/1986) - [Wanderers, Lazio, Galatasaray]
Edin DZEKO (17/03/1986) - [Manchester City, Roma, Inter, Fenerbahce]
```

This option copies all players into a LIFO stack and immediately pops them, printing the list in reverse alphabetical order to demonstrate stack behaviour.

Option 8 – Exit

```
Your choice (1-8): 8
💾 List saved to 'players_updated.txt'.
👋 Program terminating...
```

This option saves the new updated list then exits.

Time Log

Phase	Tasks Covered	Hours Spent
Design	• Reading the specification • Planning file format & menu flow	**2 h**
Implementation	• Coding Date, Player, DoublyLinkedList, MyStack • Utility classes (file loader/saver, menu helpers) • Main menu loop & I/O prompts	**10 h**
Testing / Debugging	• Unit-style tests for date validation & stack expansion • Loading sample file, add/search/delete cycles • Edge-case checks (invalid dates, empty list)	**1 h**
Documentation	• Writing code comments • Preparing report sections & screenshots	**2 h**
Total		**15 hours**

Class Explanations

Date.java

```java
import java.util.Scanner;

/**
 * Represents a date (month, day, and year).
 */
public class Date {
    private String month;
    private int day;
    private int year; // a four digit number.

    public Date() {
        month = "January";
        day = 1;
        year = 1000;
    }

    public Date(int monthInt, int day, int year) {
        setDate(monthInt, day, year);
    }

    public Date(String monthString, int day, int year) {
        setDate(monthString, day, year);
    }

    public Date(int year) {
        setDate(1, 1, year);
    }

    public Date(Date aDate) {
        if (aDate == null) {
            throw new IllegalArgumentException("Date object is null.");
        }
        month = aDate.month;
        day = aDate.day;
        year = aDate.year;
    }

    public void setDate(int monthInt, int day, int year) {
        if (dateOK(monthInt, day, year)) {
            this.month = monthString(monthInt);
            this.day = day;
            this.year = year;
        } else {
```

The **Date** class represents a calendar date with fields for month, day, and year. It provides multiple constructors and setters for initializing or changing the date, including validation to ensure dates are valid (for example, it checks month ranges and leap years). The default constructor sets the date to January 1, 1000. Methods like setDate(int monthInt, int day, int year) and setDate(String monthString, int day, int

year) use helper functions (dateOK, isLeapYear, monthString) to verify the date before setting it. For example, setDate(int, int, int) checks validity and either updates the fields or throws an exception if the date is out of range:

```
public void setDate(int monthInt, int day, int year) {
    if (dateOK(monthInt, day, year)) {
        this.month = monthString(monthInt);
        this.day = day;
        this.year = year;
    } else {
        throw new IllegalArgumentException("Invalid date.");
    }
}
```

The class also provides getters (getMonth(), getDay(), getYear()) and a toString() method to format the date. Copy constructors and mutator methods ensure that the Date object maintains valid values and does not expose internal inconsistencies.

DoublyLinkedList.java

```
import java.util.NoSuchElementException;

/**
 * Doubly linked list:
 * - Stores data of type Player.
 * - With head and tail references,
 *   --> traversal from start (A→Z) and end (Z→A) is possible.
 */
public class DoublyLinkedList {

    /* ---------- Inner Node Class ---------- */
    private class Node {
        Player data;
        Node prev;
        Node next;

        Node(Player data) {
            this.data = data;
        }
    }

    /* ---------- Fields ---------- */
    private Node head;   // head of the list (smallest: front)
    private Node tail;   // tail of the list (largest: back)
    private int size;   // node count (for tracking)

    /* ---------- Basic Queries ---------- */
    public boolean isEmpty() { return size == 0; }
    public int size()        { return size; }

    /* ---------- Sorted Insertion (alphabetical) ---------- */
    public void insertSorted(Player p) {
        Node newNode = new Node(p);

        // 1) If the list is empty
        if (isEmpty()) {
            head = tail = newNode;
        }
        // 2) Insert before head
        else if (p.compareTo(head.data) <= 0) {
            newNode.next = head;
            head.prev    = newNode;
            head         = newNode;
```

The **DoublyLinkedList** class implements a sorted doubly linked list of Player objects. It uses an inner Node class, where each node stores a Player object and references to the previous and next nodes. The list maintains head and tail references for fast access to the beginning and end of the list, as well as a size counter for tracking the number of nodes.

Node (Inner Class)

Each Node in the list holds:

- Player data: the stored player.
- Node prev: reference to the previous node.
- Node next: reference to the next node.

These links allow the list to be traversed forwards and backwards. The private constructor of Node simply sets the data field.

Fields and Basic Queries

- head and tail refer to the first and last nodes (alphabetically smallest and largest players).
- size keeps track of how many players are in the list.
- Methods isEmpty() and size() provide basic queries on the list's state.

These fields and methods support checking whether the list has elements and how many it contains, which is useful before inserting or removing elements.

Sorted Insertion

The insertSorted(Player p) method adds a player in alphabetical order (A → Z) by name and surname. Its logic:

1. **Empty list:** If the list is empty, the new node becomes both head and tail.
2. **Before head:** If the new player should come before the current head (compareTo(head) <= 0), insert at the front.
3. **After tail:** If the new player should come after the current tail (compareTo(tail) >= 0), insert at the end.
4. **Middle insertion:** Otherwise, traverse from the head until finding the correct position, then insert the new node between two existing nodes.

```
// 1) If the list is empty
if (isEmpty()) {
    head = tail = newNode;
}
// 2) Insert before head
else if (p.compareTo(head.data) <= 0) {
    newNode.next = head;
    head.prev    = newNode;
    head         = newNode;
}
// 3) Insert after tail
else if (p.compareTo(tail.data) >= 0) {
    tail.next    = newNode;
    newNode.prev = tail;
    tail         = newNode;
}
// 4) Insert in between
else {
    Node cur = head.next;
    while (cur != null && p.compareTo(cur.data) > 0) {
        cur = cur.next;
    }
    // cur is now the node to the right of the new node
    Node left = cur.prev;
    left.next = newNode;
    newNode.prev = left;
    newNode.next = cur;
    cur.prev = newNode;
}
size++;
```


This ensures the list remains sorted by name (and by surname if names are equal). The compareTo method of Player handles the alphabetical comparison.

Search and Deletion

The search(String name, String surname) method traverses from head to tail looking for a player with the matching first name and surname. If found, it returns the Player; otherwise it returns null.

```
/* ---------- Search ---------- */
public Player search(String name, String surname) {
    Node cur = head;
    while (cur != null) {
        if (cur.data.getName().equalsIgnoreCase(name) &&
            cur.data.getSurname().equalsIgnoreCase(surname))
            return cur.data;              // found
        cur = cur.next;
    }
    return null; // not found
}
```

The delete(String name, String surname) method similarly searches for the node and removes it if found. It handles four cases:

- **Single node:** If the list has only one node (head == tail), setting both to null.
- **At head:** If deleting the head, move head to head.next.
- **At tail:** If deleting the tail, move tail to tail.prev.
- **In middle:** Bypass the node by linking its previous node to its next node.

After removing the node (if found), it decrements size and returns true. If no matching player is found, it returns false. This ensures proper reconnection of the adjacent nodes in all cases.

```java
/* ---------- Deletion ---------- */
public boolean delete(String name, String surname) {
    Node cur = head;
    while (cur != null) {
        if (cur.data.getName().equalsIgnoreCase(name) &&
            cur.data.getSurname().equalsIgnoreCase(surname)) {

            // Single node
            if (head == tail) {
                head = tail = null;
            }
            // Node at head
            else if (cur == head) {
                head = head.next;
                head.prev = null;
            }
            // Node at tail
            else if (cur == tail) {
                tail = tail.prev;
                tail.next = null;
            }
            // Node in middle
            else {
                cur.prev.next = cur.next;
                cur.next.prev = cur.prev;
            }
            size--;
            return true; // deleted
        }
        cur = cur.next;
    }
    return false; // not found
}
```

Printing and Iteration

The methods printAscending() and printDescending() print all players from A → Z or Z → A by traversing the list from head to tail or vice versa. If the list is empty, they print a warning message. The class also provides an Iterator inner class to allow external iteration in A → Z order. The iterator has hasNext(), next(), and restart() methods for convenience, enabling other code to loop through the list without managing nodes directly.

Overall, **DoublyLinkedList** ensures players remain sorted by name, provides fast insertion in order, efficient traversal from both ends, and straightforward search/delete operations. It forms the core data structure for storing player records.

Main Class

The **Main** class contains the main method and provides a text-based menu interface. It uses a Scanner to read user input and a DoublyLinkedList to store players. The menu options (1–8) correspond to different operations:

- **1) Create list from file:** Uses PlayerFileLoader to build the list from a file (default "players.txt").
- **2) Add player:** Calls PlayerInputUtil to read a new player's details and insert it into the list in sorted order.
- **3) Show player info:** Uses PlayerQueryUtil to search and display a player's details by first name and surname.
- **4) Delete player:** Calls PlayerDeleteUtil to remove a player by name and surname from the list.
- **5) Print A → Z:** Uses PlayerPrintUtil.printAscending to list all players in ascending (alphabetical) order.
- **6) Print Z → A:** Uses PlayerPrintUtil.printDescending to list all players in descending order.
- **7) Copy to stack and print:** Calls PlayerStackUtil.copyListToStackAndPrint, which copies players into a MyStack and prints them in LIFO order.
- **8) Exit and save:** Uses PlayerFileSaver.saveListToFile to write the list back to a file (default "players_updated.txt") and then exits.

Each menu action either performs an operation on the linked list or delegates to a utility class. The main loop repeats until the user chooses to exit, ensuring continuous interaction and updates to the player list.

MyStack Class

The **MyStack** class is a simple stack implementation using an array of Object. It has:

- Object[] data: the internal array storage.
- int top: index of the top element (-1 when empty).
- int capacity: current capacity of the array.

Key methods:

- push(Object item): Adds an item to the top of the stack. If the array is full (top == capacity - 1), it calls expandCapacity() to double the array size. It throws an exception if a null item is pushed.
- pop(): Removes and returns the top element, throwing an exception if the stack is empty. It also nulls out the array slot to avoid memory leaks.
- isEmpty(): Returns true if the stack has no elements (top == -1).

A helper method expandCapacity() doubles the capacity, creates a new larger array, and copies existing elements. This stack is used in menu option 7 to hold players temporarily and print them in reverse order (stack LIFO behavior).

Player Class

The **Player** class represents a football player with:

- String name and String surname.
- Date birthDate (using the Date class).
- ArrayList<String> clubs: a list of clubs the player has played for, in insertion order.

Features:

- The full-argument constructor checks that name and surname are not null or empty, sets the birthDate (making a deep copy via new Date(birthDate)), and copies the clubs list into a new ArrayList.
- A copy constructor creates a deep copy of another Player, including a new Date object and a new list of clubs.
- Getters for name, surname, and birthDate (returns a new Date copy for safety), and getClubs() returns an unmodifiable list to prevent external modification.
- addClub(String clubName): Adds a club if it is not null/empty and not already in the list (avoiding duplicates).
- compareTo(Player other): Defines alphabetical sorting first by name (case-insensitive) and then by surname. This is used by the linked list to keep players sorted.

- Overridden equals() and hashCode() for comparing players by name, surname, birth date, and clubs list.
- toString(): Formats the player as, for example, "Fernando MUSLERA (16/06/1986) - [Wanderers, Lazio, Galatasaray]", making it easy to display player information.

In summary, Player encapsulates all the data about a player and provides the necessary operations to manage and compare players.

Player File Utilities

PlayerFileLoader

The **PlayerFileLoader** class has a static method buildListFromFile(String fileName). It reads a CSV file where each line has the format:

bash

KopyalaDüzenle

firstName,surname,dd/MM/yyyy,club1,club2,...

For each non-empty line, it splits by commas, trims whitespace, and parses the date (dd/MM/yyyy). It collects the club names into a list of strings. Then it constructs a Player object with this data and calls list.insertSorted(p) to insert it into a new DoublyLinkedList in alphabetical order. If a line is invalid (fewer than 4 parts), it skips that line. The method throws an IOException if file reading fails. When done, it returns the sorted list of players.

PlayerFileSaver

The **PlayerFileSaver** class has a static method saveListToFile(DoublyLinkedList list, String fileName). It writes the players to a file in CSV format (the same format as the input file). It uses the list's iterator (ascending order) and for each Player, builds a string containing the name, surname, and birth date fields separated by commas. Then it appends each club, separated by commas as well. Each player is written as one line. This is used when the program exits (menu option 8) to save the possibly updated list to a new file (players_updated.txt by default).

Player Input and Output Utilities

PlayerInputUtil

The **PlayerInputUtil** class handles adding a new player from keyboard input (menu option 2). It prompts the user for:

1. **First Name:** It loops until the user provides a non-empty string.
2. **Last Name:** Similarly ensures a non-empty string.

3. **Birth date:** The user must enter a string in dd/mm/yyyy format. The code splits the string by /, parses integers for day, month, and year, and then tries to create a new Date(month, day, year). If parsing fails or the date is invalid, it prompts again.
4. **Clubs:** The user enters club names separated by commas. The code splits this line by commas, trims whitespace, and adds each non-empty club name to a list.

After collecting valid inputs, it creates a new Player(name, surname, birthDate, clubs) and calls list.insertSorted(newPlayer) to add it in the correct position. Finally, it prints a confirmation message with the new player's details.

PlayerPrintUtil

The **PlayerPrintUtil** class provides static methods for printing the player list:

- printAscending(DoublyLinkedList list): If the list is empty, it warns the user; otherwise it prints a header and calls list.printAscending() to display each player A → Z.
- printDescending(DoublyLinkedList list): Similar, but calls list.printDescending() to display players Z → A.

These utilities ensure consistent formatting for showing all records in either order.

Player Query Utilities

PlayerQueryUtil

The **PlayerQueryUtil** class supports menu option 3 (searching for a player). It prompts the user for a first name and surname, then calls list.search(name, surname). If a matching player is found, it prints “Player found:” followed by the player’s toString() output. If not found, it prints a message indicating the player is not in the list. This utility isolates the search prompt and output formatting.

PlayerDeleteUtil

The **PlayerDeleteUtil** class supports menu option 4 (deleting a player). It first checks if the list is empty and warns if so. Otherwise, it prompts the user for the first name and surname of the player to delete. It then calls list.delete(name, surname). If delete returns true, it prints a success message; if false, it prints that the player was not found. This cleanly handles the user interaction for removal and reporting the result.

PlayerStackUtil

The **PlayerStackUtil** class handles menu option 7 (copying the list to a stack and printing). It performs these steps:

1. Checks if the list is empty; if so, it prints a warning.

2. Creates a new MyStack with an initial capacity equal to list.size().
3. Iterates through the DoublyLinkedList from head to tail (using the provided iterator) and pushes each Player onto the stack.
4. Pops each player from the stack and prints it. Because a stack is LIFO, this results in printing the players from Z → A (reverse of the list order).

This utility demonstrates using the custom stack class with the list data